

Software Requirements Specification (SRS)

Privacy-Preserving Inverted Index Search Engine (Benchmarking Hash Map vs. Trie)

Document Version 1.0 | IEEE 830 Standard Adaptation

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional, structural, and research specification for the **Privacy-Preserving Inverted Index Search Engine**. The system provides localized, offline full-text document indexing and boolean search functionality over raw text corpora while empirically benchmarking two foundational index structures: Hash Maps and Trie (Prefix) Trees.

1.2 Scope

The target product is a lightweight command-line and programmatic search framework. Key capabilities include:

- Automated local directory ingestion and text sanitization (lowercasing, punctuation stripping, stop-word filtering).
- Dual inverted index construction (Hash Map vs. Trie Prefix Tree).
- Boolean search execution (AND, OR, NOT) combined with Term Frequency-Inverse Document Frequency (TF-IDF) relevancy ranking.
- Empirical execution profiling tracking query latency (microseconds) and RAM consumption (KB) across varying corpus sizes.
- Strict zero-network execution guarantee ensuring total user privacy.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
Inverted Index	A data structure mapping distinct words to the list of documents containing them

Term / Acronym	Definition
	(posting lists).
TF-IDF	Term Frequency - Inverse Document Frequency; a numerical statistic indicating word relevance in a document collection.
Trie	A tree-like data structure used to store associative data structures where keys are usually strings.
Posting List	A list of document IDs and term frequencies corresponding to a specific key in an inverted index.

2. Overall Description

2.1 Product Perspective

The system operates entirely offline as a self-contained local processing tool. It acts as an intermediary layer between local document storage and user query inputs, providing instantaneous search responses without communicating with external services or remote cloud instances.

2.2 Product Functions

- Ingest and preprocess document files (ASCII/UTF-8 encoded plain text and Markdown files).
- Construct and maintain in-memory Hash Map and Trie inverted index structures.
- Execute single-keyword, multi-keyword, and boolean constraint queries.
- Compute relative TF-IDF scores for query matches and display ranked search results.
- Execute automated performance micro-benchmarks and output timing and memory logs.

2.3 User Classes and Characteristics

- **Software Developer / Researcher:** Utilizes the software via CLI or imports modules to evaluate data structure metrics and test performance scaling.
- **End User:** Uses the search utility to locate local documents via rapid boolean and term

queries without exposing document contents to online tools.

2.4 Operating Environment

- **OS:** Cross-platform support (Linux, macOS, Windows). Fully compatible with POSIX environments.
- **Runtime:** Python 3.10+ runtime environment.
- **Hardware:** Minimum 512 MB RAM, dual-core CPU. Recommended 4 GB RAM for large text corpora indexing (up to 100 MB).

3. System Features & Functional Requirements

3.1 Text Preprocessing & Ingestion Module

Description: Reads raw text files from target local directories and converts unstructured text into clean token streams.

- **FR-1.1 (File Parsing):** System shall recursively read .txt and .md files from a user-specified root folder.
- **FR-1.2 (Case Normalization):** System shall convert all text characters to lowercase.
- **FR-1.3 (Punctuation Stripping):** System shall strip non-alphanumeric characters while preserving word boundaries.
- **FR-1.4 (Stop-Word Removal):** System shall filter out predefined high-frequency stop-words (e.g., "the", "is", "at") using an $O(1)$ set lookup.

3.2 Indexing Module

Description: Generates inverted index structures mapping tokens to document posting lists.

- **FR-2.1 (Hash Map Indexing):** System shall construct a dictionary-based index storing terms as keys and lists of (doc_id, term_frequency) tuples as values.
- **FR-2.2 (Trie Indexing):** System shall construct a character-based prefix tree where terminal leaf nodes contain the posting list for the full word path.
- **FR-2.3 (Dynamic Index Update):** System shall support inserting new documents into both index structures without requiring a full re-index rebuild.

3.3 Query Processing & Relevancy Engine

Description: Receives raw search queries, resolves document matches, and ranks outputs.

- **FR-3.1 (Boolean Query Resolution):** System shall support boolean operations:

- AND: Intersects posting lists of term keys.
- OR: Unions posting lists of term keys.
- NOT: Subtracts posting list of excluded terms from result set.
- **FR-3.2 (Prefix/Autocomplete Search):** System shall leverage the Trie index to return matches for partial prefix strings (e.g., query "algo" matches "algorithm").
- **FR-3.3 (TF-IDF Ranking):** System shall compute document scores using the formula:

$$TF(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}, \quad IDF(t, D) = \log\left(\frac{|D|}{|\{d \in D : t \in d\}| + 1}\right)$$

Match output shall be sorted by descending total score.

3.4 Research Benchmarking Engine

Description: Collects quantitative data to evaluate Hash Map vs. Trie efficiency.

- **FR-4.1 (Latency Measurement):** System shall measure index construction time and query response latency in microseconds (μs).
- **FR-4.2 (Memory Tracking):** System shall measure peak memory overhead (RAM in KB/MB) for both index data structures.
- **FR-4.3 (Benchmark Reporting):** System shall export metric logs in JSON/CSV formats for plotting performance curves across scaling text sizes (1 MB to 100 MB).

4. Non-Functional Requirements (NFR)

4.1 Performance Requirements

- **NFR-1.1 (Query Latency):** Average lookup response time for single-term queries shall not exceed 50 ms on document collections up to 50 MB .
- **NFR-1.2 (Index Build Throughput):** System shall process and index raw text at a minimum throughput of 5 MB/sec .

4.2 Security & Privacy Requirements

- **NFR-2.1 (Network Isolation):** System shall strictly refrain from opening network sockets, HTTP requests, or external IPC calls.
- **NFR-2.2 (Local Data Persistence):** All temporary memory buffers and generated index files shall remain strictly inside the user-designated directory.

4.3 Software Quality & Maintainability

- **NFR-3.1 (Test Coverage):** Core parsing, indexing, and ranking code modules shall achieve

\$>90\%\$ branch test coverage verified via unit test suites.

- **NFR-3.2 (Modular Architecture):** Components shall maintain clean decoupling, enabling independent swapping of index implementations via abstract interface classes.

5. System Design Architecture

5.1 Modular Folder Structure

```
privacy-search-engine/
├── docs/           # SRS documentation, UML diagrams, report source
├── src/            # Core source code
│   ├── indexer/    # preprocessor.py, hash_index.py, trie_index.py
│   ├── search/     # query_engine.py, tfidf_ranker.py
│   └── utils/       # benchmark_runner.py, memory_tracker.py
├── tests/          # Unit testing framework
│   ├── test_preprocessor.py
│   ├── test_indexer.py
│   └── test_tfidf.py
├── data/           # Corpus datasets (.txt files)
├── README.md       # General repository overview
└── requirements.txt # Project dependencies (pytest, matplotlib, psutil)
```

6. Unit Testing Strategy

Module Name	Test Scope	Validation Strategy
test_preprocessor.py	Text Sanitization	Assert correct lowercasing, handling of empty files, punctuation removal, and stop-word filtering.
test_indexer.py	Hash Map & Trie Index Integrity	Verify that adding sample documents creates matching posting lists across both Hash Map and Trie implementations.

Module Name	Test Scope	Validation Strategy
test_tfidf.py	Math Relevancy Scoring	Mock fixed 2-document corpus and assert calculated TF-IDF float values match manual calculations.